

CFD 2025-26 – Lab 05 – solution

The full code for this lab is provided in the notebook `CFDlab05.ipynb`. This code contains many instructions related to the direct use of the PETSc interface to handle matrices and vectors and to solve linear systems. Inline comments in the code already contain the documentation on the meaning and syntax of many definitions and commands, thus in this report we do not explain every single line. Further information can be found at <https://www.firedrakeproject.org/petsc-interface.html> and in the documentation of PETSc and Firedrake.

We introduce here the penalty method for the imposition of Dirichlet boundary conditions, for a generic problem

$$\begin{cases} -\Delta \mathbf{u} + \nabla p = \mathbf{0} & \text{in } \Omega, \\ \operatorname{div} \mathbf{u} = 0 & \text{in } \Omega, \\ \mathbf{u} = \mathbf{g} & \text{on } \Gamma_D, \\ \mathbf{n} \cdot \nabla \mathbf{u} - p \mathbf{n} = \mathbf{h} & \text{on } \Gamma_N = \partial\Omega \setminus \Gamma_D, \end{cases} \quad (1)$$

with given boundary data functions $\mathbf{g}, \mathbf{h}$. As seen in the previous laboratories, this problem should consider as a test space the subspace $V_0 = [H_{\Gamma_D}^1(\Omega)]^2$ to enforce Dirichlet boundary conditions *in a strong sense*. Using the penalty method, instead, we impose also Dirichlet boundary conditions *weakly*. This is possible by introducing a perturbation of the Dirichlet condition: for a small $\varepsilon > 0$, we replace $\mathbf{u} = \mathbf{g}$ by

$$\mathbf{u} = \mathbf{g} - \varepsilon(\nabla \mathbf{u} - p \mathbf{I}) \mathbf{n} \quad \text{on } \Gamma_D.$$

Reordering these terms, we can write the condition as

$$-(\nabla \mathbf{u} - p \mathbf{I}) \mathbf{n} = \frac{1}{\varepsilon}(\mathbf{u} - \mathbf{g}) \quad \text{on } \Gamma_D. \quad (2)$$

Replacing the Dirichlet condition with (2), the weak formulation of (1) reads as follows:

$$\begin{aligned} &\text{Find } (\mathbf{u}, p) \in V \times Q = [H^1(\Omega)]^2 \times L^2(\Omega) \text{ s.t.} \\ &a(\mathbf{u}, \mathbf{v}) + b(\mathbf{v}, p) - b(\mathbf{u}, q) = L(\mathbf{v}), \quad \text{for all } (\mathbf{v}, q) \in V \times Q, \\ &\text{where} \\ &a(\mathbf{u}, \mathbf{v}) := \int_{\Omega} (\nabla \mathbf{u} : \nabla \mathbf{v}) \, d\mathbf{x} + \frac{1}{\varepsilon} \int_{\Gamma_D} \mathbf{u} \cdot \mathbf{v} \, ds. \\ &b(\mathbf{v}, q) := - \int_{\Omega} \operatorname{div} \mathbf{v} \, q \, d\mathbf{x}, \\ &L(\mathbf{v}) := \int_{\Gamma_N} \mathbf{h} \cdot \mathbf{v} \, ds + \frac{1}{\varepsilon} \int_{\Gamma_D} \mathbf{g} \cdot \mathbf{v} \, ds. \end{aligned}$$

We notice that the full H^1 space is used as trial and test space for velocity, and the right-hand side functional L contains a contribution from the Dirichlet data $\mathbf{g}$.

In the actual implementation of this method, we are going to use an extremely small value $\varepsilon = 10^{-20}$. In this way, the perturbation introduced in the problem is below the order of

machine precision, but the imposition of the conditions is extremely effective. Indeed, denoting by $\bar{i}$ an index corresponding to a velocity d.o.f. lying on the Dirichlet boundary, the $\bar{i}$ -th row of the algebraic formulation of the FEM for the problem is dominated by the terms multiplied by the very large number $\frac{1}{\varepsilon}$, thus basically imposing

$$\int_{\Gamma_D} \mathbf{u} \cdot \boldsymbol{\varphi}_{\bar{i}} = \int_{\Gamma_D} \mathbf{g} \cdot \boldsymbol{\varphi}_{\bar{i}}.$$

Exercise 1

- 1 Using the penalty method, the weak formulation of the problem reads as follows:

Find $(\mathbf{u}, p) \in V \times Q$ s.t. $a(\mathbf{u}, \mathbf{v}) + b(\mathbf{u}, q) + b(\mathbf{v}, p) = L(\mathbf{v})$, for all $(\mathbf{v}, q) \in V \times Q$,

where

$$a(\mathbf{u}, \mathbf{v}) := \int_{\Omega} (\nabla \mathbf{u} : \nabla \mathbf{v}) \, d\mathbf{x} + \frac{1}{\varepsilon} \int_{\partial\Omega} \mathbf{u} \cdot \mathbf{v} \, ds.$$

$$b(\mathbf{v}, q) := - \int_{\Omega} \operatorname{div} \mathbf{v} \, q \, d\mathbf{x},$$

$$L(\mathbf{v}) := \frac{1}{\varepsilon} \int_{\Gamma_{up}} \mathbf{i} \cdot \mathbf{v} \, ds,$$

where $V = [H^1(\Omega)]^d$, $Q = L^2(\Omega)$ are the “full” spaces, even though we are considering a fully Dirichlet problem: the weak imposition of the conditions prevents (in principle) the non-uniqueness of pressure. Actually, since $1/\varepsilon$ is extremely high, the condition number of the algebraic system of the FEM is very high, which in turns is basically equivalent to say that the algebraic system is *almost* singular. However, for this lab we will neglect this issue.

- 2 The matrices A, B and the right-hand side $\mathbf{F}$ corresponding to the FEM can be assembled hinging upon the definition of the corresponding forms and functional, as reported in the code, and then using the `assemble` function.

```
A_fd = assemble(a) # firedrake Matrix.
# No Dir. bcs argument passed: penalty method.
A = A_fd.M.handle # PETSc Mat object underlying the firedrake Matrix
minusB = assemble(minusb).M.handle # PETSc Mat for -B
F = assemble(L)
```

The `assemble` function returns a matrix/vector of the type `firedrake.Matrix/Vector`. We can access directly the underlying `PETSc.Mat` contained in a `firedrake.Matrix` through its attribute `M.handle`.

- 3 In this lab, we are going to work directly with the PETSc objects underlying in Firedrake. The advantages are that PETSc provides a useful function to define the Schur complement of a block matrix, and that being able to access the linear algebra objects allows to have further control on our implementation.

By block-wise Gaussian elimination and forward/backward substitution, we get

$$\begin{aligned} \begin{bmatrix} A & B^T \\ -B & 0 \end{bmatrix} \begin{bmatrix} \mathbf{U} \\ \mathbf{P} \end{bmatrix} &= \begin{bmatrix} \mathbf{F} \\ \mathbf{0} \end{bmatrix} \Leftrightarrow \begin{bmatrix} I & 0 \\ -BA^{-1} & I \end{bmatrix} \begin{bmatrix} A & B^T \\ 0 & BA^{-1}B^T \end{bmatrix} \begin{bmatrix} \mathbf{U} \\ \mathbf{P} \end{bmatrix} = \begin{bmatrix} \mathbf{F} \\ \mathbf{0} \end{bmatrix} \\ \Leftrightarrow \begin{bmatrix} I & 0 \\ -BA^{-1} & I \end{bmatrix} \begin{bmatrix} \boldsymbol{\zeta} \\ \boldsymbol{\chi} \end{bmatrix} &= \begin{bmatrix} \mathbf{F} \\ \mathbf{0} \end{bmatrix} \text{ and then } \begin{bmatrix} A & B^T \\ 0 & BA^{-1}B^T \end{bmatrix} \begin{bmatrix} \mathbf{U} \\ \mathbf{P} \end{bmatrix} = \begin{bmatrix} \boldsymbol{\zeta} \\ \boldsymbol{\chi} \end{bmatrix} \\ \Leftrightarrow \boldsymbol{\zeta} = \mathbf{F} \text{ then } -BA^{-1}\boldsymbol{\zeta} + \boldsymbol{\chi} = \mathbf{0} &\text{ then } BA^{-1}B^T\mathbf{P} = \boldsymbol{\chi} \text{ then } A\mathbf{U} + B^T\mathbf{P} = \boldsymbol{\zeta} = \mathbf{F}. \end{aligned}$$

Regarding matrix A , we need to “invert” it multiple times, to define the right-hand side vector $\boldsymbol{\chi} = BA^{-1}\mathbf{F}$ of the Schur complement system, the Schur complement itself $S = BA^{-1}B^T$, and the velocity reconstruction $\mathbf{U} = A^{-1}(\mathbf{F} - B^T\mathbf{P})$. Actually inverting is an extremely costly operation, therefore, in place of A^{-1} , we construct a linear solver associated with the matrix A . Specifically, we construct a `PETSc.KSP` solver, by means of the following instructions:

```
petsc_options = PETSc.Options()

ksp_A = PETSc.KSP().create()
ksp_A.setOperators(A)
petsc_options['ksp_type'] = 'preonly'
petsc_options['ksp_type'] = 'lu'
ksp_A.setFromOptions()      # use the info stored in petsc_options
ksp_A.setUp()               # finalize the definition of the solver
```

Notice that `petsc_options` contains the same kind of options that we passed to `LinearVariationalSolver` in the previous lab. Indeed, all linear algebra settings defined in Firedrake are passed to the PETSc linear algebra backend through the object `PETSc.Options()`, of which `petsc_options` is a proxy. Throughout this lab, focused on the Schur complement system, we consider a direct (LU) solver for A : other choices may be possible, to improve computational efficiency.

For the definition of the Schur complement S , we want to avoid assembling A^{-1} (for the same reason mentioned above), so we use the method `createSchurComplement` of the class `PETSc.Mat`:¹

```
createSchurComplement(A00, pc_A00, A01, A10, A11)
```

which defines a matrix-free operator that does not need to explicitly assemble the matrix S , but only to encode the matrix-vector multiplication corresponding to $S\mathbf{x}$ for a certain input vector $\mathbf{x}$: indeed, this is the only feature needed to apply a Krylov method for the solution of the system associated to S . The syntax of the function is quite self-explaining, considering the monolithic matrix of the system as a block matrix of type $\begin{bmatrix} A_{00} & A_{01} \\ A_{10} & A_{11} \end{bmatrix}$: the return value corresponds to an operator S such that $S\mathbf{x} = A_{11}\mathbf{x} - A_{01}A_{00}^{-1}A_{01}\mathbf{x}$, where A_{00}^{-1} is replaced by a KSP solver with matrix A_{00} and preconditioner `pc_A00`. Since we want to use a direct method over A_{00} , we can pass the same matrix A also as `pc_A00`.

¹See <https://petsc.org/release/petsc4py/reference/petsc4py.PETSc.Mat.html> for the documentation on the class `PETSc.Mat` and its attributes and methods.

Analogously to the stiffness matrix, we also define a KSP solver for the Schur complement S , by which we will be able to solve the problem.

Then, the solution of the problem is obtained via the following steps, where $\mathbf{y} \in \mathbb{R}^{\mathbf{V.dim}()}$ and $\mathbf{z}, \mathbf{w} \in \mathbb{R}^{\mathbf{Q.dim}()}$:

1. construct the right-hand side vector χ :
 - a. solve the system $A\mathbf{y} = \mathbf{F}$ by means of `ksp_A`;
 - c. compute $\chi = B\mathbf{y}$ by the `mult` method of `B`, and store it into a vector $\mathbf{w}$.
2. solve the system $S\mathbf{z} = \mathbf{w}$ by means of `ksp_S` and store it in $\mathbf{P}$;
3. reconstruct velocity:
 - a. compute $\mathbf{y} = B^T \mathbf{z}$ by the `mult` method of `Bt`;
 - b. solve the system $A\mathbf{U} = \mathbf{y}$ by means of `ksp_A`.

- 4 We create a form `pc_S_form` encoding the preconditioner, similarly to what we have done in the previous lab. Then, we assemble a `PETSc.Mat` object out of it and use it in the definition of the KSP solver for S :

```
pc_S_mat = assemble(pc_S_form).MM.handle
...
ksp_S.setOperators(S, pc_S_mat)
...
```

To implement the different preconditioners, we just need to modify the matrix `pc_S_mat` before passing it to `ksp_S.setOperators(S, pc_S_mat)`: see `CFDlab04.ipynb` for the details. In particular, to create the lumped mass matrix $\tilde{M}_p$, we notice that its diagonal $[\sum_{k=0}^{N_p} (M_p)_{ik}]_i = 1^{N_p}$ can be re-written as a vector $\mathbf{m} = M\mathbf{1}$, where $\mathbf{1}$ is a vector with all the entries equal to 1.

Solving the problem with different preconditioning choices, we notice that the diagonal preconditioners are very effective in reducing the number of linear solver iterations required to reach convergence, w.r.t. the non-preconditioned case.

Exercise 2

We introduce a set of time nodes $\{t^n\}_{n=0}^{N_T}$, with $t^0 = 0, t^{N_T} = 1, t^{n+1} - t^n = \Delta t$ and a mesh $\mathcal{T}_h$ for the domain Ω . The fully discrete problem associated to the unsteady system, using the implicit Euler scheme in time and the FEM in space, and imposing Dirichlet conditions by the penalty method, reads as follows:

Given $\mathbf{u}_h^0 = \mathbf{0}$, for each $n = 0, \dots, N_T - 1$, find $(\mathbf{u}^{n+1}, p^{n+1}) \in V \times Q = [H^1(\Omega)]^2 \times L^2(\Omega)$ s.t.

$$m(\mathbf{u}^{n+1}, \mathbf{v}) + a(\mathbf{u}^{n+1}, \mathbf{v}) + b(\mathbf{v}, p^{n+1}) + b(\mathbf{u}^{n+1}, q) = L^{n+1}(\mathbf{v}) \quad \forall (\mathbf{v}, q) \in V \times Q,$$

where a, b are defined as in Exercise 1 and

$$m(\mathbf{u}, \mathbf{v}) = \frac{1}{\Delta t} \int_{\Omega} \mathbf{u} \cdot \mathbf{v} \, d\mathbf{x}, \quad L^{n+1}(\mathbf{v}) = m(\mathbf{u}^n, \mathbf{v}) + \frac{1}{\varepsilon} \int_{\Gamma_{up}} t^{n+1} v_y \, ds.$$

Denoting by A, M the stiffness and mass matrix in velocity and introducing $\mathcal{A} = A + M$, the structure of the algebraic system to be solved at each time step is analogous to the one of Exercise 1, with

$$\begin{bmatrix} A_{00} & A_{01} \\ A_{10} & A_{11} \end{bmatrix} = \begin{bmatrix} \mathcal{A} & B^T \\ -B & 0 \end{bmatrix}.$$

Therefore, we replace the stiffness solver `ksp_A` with a `ksp_MA` that encodes the inversion of $\mathcal{A}$ by means of the LU method. Then we can directly use the functions `define_stokes_schur_solver`, `solve_problem` defined in Exercise 1, as follows:

```
...
# Create the correct Schur complement.
S, ksp_S = define_stokes_schur_solver(A, B, Bt, pc_form)

# Solve time-dependent problem.
...
for t in np.arange(t0+dt, T, dt):
    with ... , F.dat.vec_ro as vecF, ...: # Assemble the current rhs of the
        system.
        vecF.copy(tmp_u) # deep copy!
        tmp_u *= t
        M.multAdd(vecUold, tmp_u, rhs) # rhs = M*vecUold + tmp_u

    # Advance in time: solve the problem with M+A in place of A
    uh, ph = solve_problem(V, Q, ksp_MA, B, Bt, rhs_u_fun, ksp_S)

    u0 = uh # update the old solution
```

We point out that, in order to prevent undesired overwriting, in the assembly of the rhs of the system we create a *deep copy* of the vector corresponding to $\bar{L}(\mathbf{v})$ by means of the method `copy` of `numpy.arrays`. Indeed, if we had wrote `tmp_u = vecF`, only a *shallow copy* would have been performed, namely only a pointer would have been created, pointing to the same memory area of `vecF`.

We also notice the use of the NumPy command `arange`, which creates a range of floating-point numbers with a given step: the standard python function `range` only works with integers.

Solving this problem, we can see that the fluid in the cavity, initially at rest ($\mathbf{u}^0 = \mathbf{0}$), accelerates with a generation of a progressively faster cavity vortex.